

COMPSYS302: Python

2. Data types and Operations

Ho Seok AHN

hs.ahn@auckland.ac.nz

2.1 Lines and Indentation

□ Lines

- A Python program is composed of a sequence of logical lines.
- Comment
 - A pound sign (#) that is not inside a string literal begins a comment.
 - All characters after the # and up to the physical line end are part of the comment.
 - Python interpreter ignores them.
 - `"""xxx"""` is for multiline comment.
 - Ex) 2_01_Comments.py

```
a = "'#' is for comment"  
print (a)
```

(result)

'#' is for comment

```
#b = "this is comment"  
print (b)
```

(result)

Traceback (most recent call last):

File "COMPSYS302\ 2_1_Comments.py", line 2, in <module>

'#' is for comments

print (b)

NameError: name 'b' is not defined

2.1 Lines and Indentation

□ Lines

▪ Semicolon

- The end of a physical line marks the end of most statements.
- Unlike in other languages, Python statements are not normally terminated with a delimiter, such as a semicolon(;).
- But you can use it.

▪ Ex) 2_02_Semicolon.py

```
a = "You do not need to use semicolon"  
b = "But you can use it";  
print (a)  
print (b)
```

(result)

You do not need to use semicolon

But you can use it

2.1 Lines and Indentation

□ Lines

- Semicolon

- When more than two statements are in one physical line, you can use semicolon.

- Ex) 2_02_Semicolon.py

- a = "You do not need to use semicolon"

- b = "But you can use it"

- print (a); print (b)

- (result)

- You do not need to use semicolon

- But you can use it

- How about this case?

- >>> print (a), (b)

- (result)

- You do not need to use semicolon

2.1 Lines and Indentation

□ Lines

▪ Backslash

- When a statement is too long to fit on a single physical line, you can join them into a logical line.
- For this, you can use a backslash (\)

▪ Ex) 2_03_Backslash.py

```
a = "When a statement is too long to fit on a single physical line,\  
you can join them into a logical line.\  
For this,\  
you can use a backslash ( \ )"  
print (a)
```

(result)

When a statement is too long to fit on a single physical line, you can join them into a logical line.

For this, you can use a backslash (\)

2.1 Lines and Indentation

□ Indentation

- Indentation is one of features of Python
 - It helps easy reading of code.
 - But, it can be the bug cannot be found easily if it is logical error.
 - It is better to use same indentation method, such as tab and space.
 - Ex) 2_04_Indentation.py

```
a = 1
if a == 1:
    result = "a is 1"
else:
    result = "a is not 1"
print (result)
```

(result)
a = 1

```
a = 1
if a == 1:
    result = "a is 1"
print (result)
print (a)
```

(result)
File "COMPSYS302\ 2_4_Indentation.py", line 5
print (a)
IndentationError: unexpected indent

2.1 Lines and Indentation

□ Identifier

▪ Identifiers

- An **Identifier** is a name used to identify a variable, function, class, module, or other object.
- An identifier starts with a letter (A to Z, a to z) or underscore (_) followed by zero or more letters, underscores, and digits (0 to 9).
- Case is significant; Lowercase and uppercase letters are distinct.
- Punctuation characters such as %, \$, and @ are not allowed in identifiers.
- Ex) 2_05_Identifiers.py

```
friend = 1; Friend = 99
```

```
print (friend)
```

```
print (Friend)
```

(result)

1

99

2.1 Lines and Indentation

□ Keyword

▪ Identifiers vs. Keywords

- Python has several **keywords**, which are identifiers that **Python reserves** for special syntactic uses.
- **Keywords** are composed of lowercase letters only.
- You cannot use **keywords** as regular identifiers.
- Examples of **keywords**:
and, as, assert, break, class, continue, def, del, elif, else, except, is, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield
- Ex) 2_06_Keyworks.py

```
for = 1  
print (for)
```

(result)

File "COMPSYS302\ 2_6_Keywords.py", line 1

```
for = 1
```

SyntaxError: invalid syntax

2.2 Data Types

□ Numbers

- The built-in number object in Python supports integers (plain and long), floating point numbers, and complex numbers.
- Integer literals can be decimal, binary, octal, or hexadecimal.
 - A **decimal literal** is represented by a sequence of digit where the first digit is non-zero.
 - Ex) 1, 23, 4567
 - To indicate a **binary literal**, use **0b** followed by a sequence of binary digits (0 and 1).
 - Ex) 0b1, 0b10, 0b11101
 - An **octal literal** is specified with a **0o** followed by a sequence of octal digits (0 to 7).
 - Ex) 0o1, 0o37, 0o667432,
 - To indicate a **hexadecimal literal**, use **0x** followed by a sequence of hexadecimal digits (0 to 9 and A to F). You can use uppercase or lowercase both.
 - Ex) 0x1, 0x17, 0xAB34, 0xabcdef12345
 - Any kind of integer literal may be followed by the letter **L or l** to denote a long integer.
 - Ex) 1L, 123L, 4567L, 01L, 037L, 0667432L, 0x1L, 0x17L, 0xAB34L

2.2 Data Types

□ Numbers

- You can convert decimal literal to various kind of integer literal.

- Ex) 2_07_Numbers.py

```
a = 38          # decimal literal
print (bin(a))   # to binary literal
print (oct(a))   # to octal literal
print (hex(a))   # to hexadecimal literal
```

(result)

0b100110

0o46

0x26

2.2 Data Types

□ Numbers

- Floating point numbers
 - A floating point literal is represented by a sequence of decimal digits that includes a decimal point (.), and exponent part (an e or E, optionally followed by + or -, followed by one or more digits), or both.
 - Ex) 0., 0.0, .0, 1., 1.0, 3.14, 314e-2
- Complex numbers
 - A complex number is made up of two floating point values, one each for the real and imaginary parts.
 - You can specify an imaginary literal as a floating point or decimal literal followed by a j or J.
 - Ex) 0j, 0.j, 0.0j, .0j, 1e0j, 1.0e0j

2.2 Data Types

□ Strings

- A string literal can be quoted or double-quoted.
- A quoted string is a sequence of zero or more characters enclosed in matching quote characters, single (') or double (").
 - Ex) 2_08_Strings.py
`'string'` , `"string"` , `"This is a 'string'"`
- The two different kinds of quotes function identically; having both allows you to include one kind of quote inside of a string specified with the other kind without needing to escape them with the backslash character (`\`).

- Ex) 2_08_Strings.py

```
a = 'I'm a student'  
print (a)
```

(result)

File "COMPSYS302\ 2_8_Strings.py", line 1

```
a = 'I'm a student'
```

SyntaxError: invalid syntax

```
a = 'I\' 'm a student'
```

```
b = "I'm a student"
```

```
print (a); print (b)
```

(result)

```
I'm a student
```

```
I'm a student
```

2.2 Data Types

□ Strings

- To make the string output on two lines, you must embed a **newline(\ n)** in the string.
- Another approach is to use a **triple-quoted string**, which is enclosed by matching triplets of quote characters (`'''` or `"""`).

- Ex) 2_08_Strings.py

```
a = " I'm a student\ n\  
  of the University of Auckland"  
b = """ I'm a student  
  of the University of Auckland"""  
c = """ I'm a student\  
  of the University of Auckland"""  
print (a); print (b); print (c)
```

(result)

```
I'm a student  
of the University of Auckland  
I'm a student  
of the University of Auckland  
I'm a student of the University of Auckland
```

2.2 Data Types

□ **Strings**

- String escape sequences

- Ex)

- \ 0 null

- \ n newline

- \ t tab

- \ v vertical tab

- \ b backspace

- \ \ backslash

- \ ' single quote

- \ " double quote

2.2 Data Types

□ Sets

- A set is an **immutable unordered** sequence of items.
- To specify a set, use a series of expressions separated by commas (,) and within braces ({}).

- Ex)

`{12345}` # set with one item

`{12, 34.56, 'hello'}` # set with three items (different types)

`{ }` # empty set

2.2 Data Types

□ Tuples

- A tuple is an **immutable ordered** sequence of items.
- To specify a tuple, use a series of expressions separated by commas (,) and within parentheses (()).
- You may optionally place a redundant comma after the last item.
- To create a tuple of one item, add a comma to the end of the expression.
- A tuple with exactly two items is often called a pair.
- An empty tuple is denoted by an empty pair of parentheses.

▪ Ex)

(12345,)	# tuple with one item – it does not matter to have redundant comma
(12, 34.56)	# tuple with two items (pair)
()	# empty tuple
tuple ('hello')	# equal to ('h', 'e', 'l', 'l', 'o')

2.2 Data Types

□ Lists

- A list is a **mutable ordered** sequence of items.
- To specify a list, use a series of expressions separated by commas (,) and within brackets ([]).
- You may optionally place a redundant comma after the last item.
- Python does not have built-in support for Arrays, but Python Lists can be used instead.
- An empty list is denoted by an empty pair of brackets.

- Ex)

[12345]	# list with one item
[12, 34.56, 'hello']	# list with three items
[]	# empty list
list('hello')	# equal to ['h', 'e', 'l', 'l', 'o']

2.2 Data Types

□ Dictionaries

- A mapping is an arbitrary collection of objects indexed by nearly arbitrary values called key.
- Python provides a single built-in mapping type, the dictionary type.
- To specify a dictionary, use a series of expressions separated by commas (,) and within braces ({}).
- You may optionally place a redundant comma after the last item.
- Each item in a dictionary is written **key:value**, where key is an expression giving the item's key and value is an expression giving the item's value.
- An empty dictionary is denoted by an empty pair of braces.

- Ex)

`{1:2, 3:4}` # dictionary with two items and integer keys

`{'x':1, 'y':2, 'z':3}` # dictionary with three items and string keys

`{ }` # empty dictionary

2.3 Operations

□ Assignment Statements

- A plain assignment statement in the simplest form has the syntax:

target = expression

- There can be **multiple targets** and equals signs (=) in a plain assignment. For example,

a = b = c = 1

- Ex) 2_09_AssignmentStatements.py

a = 1	(result)
b = c = 2	1
[d, e] = [3, 4]	2
(f, g) = (5, 6)	2
print(a), print(b), print(c), print(d), print(e), print(f), print(g)	3
	4
	5
	6

2.3 Operations

□ Numeric Operations

- Python supplies the usual numeric operations.
- You can access the parts of a complex object `z` as read-only attributes `z.real` and `z.imag`.

- Ex) `2_10_NeumericOperations.py`

```
r = 2
circle_area = 3.14 * (r ** 2)
print (circle_area)
z = 3 - 4j
print (z.real); print (z.imag)
```

(result)

12.56

3.0

-4.0

2.3 Operations

□ Sequence Operations: Strings

- Python supports a various of operations that can be applied to sequence types, including strings, lists, and tuples.
- String objects are immutable**, so attempting to rebind or delete an item or slice of a string raises an exception.

- Ex) 2_11_StringOperations.py

```
a = 'py' + 'thon'
b = 'py' 'thon'
c = 'py' * 3
print(a, b, c)
print(a[0], a[5])
print(b[:], b[0:1], b[1:4], b[:2], b[-2:], b[::-2])
```

(result)

python python pypypy

p n

python p yth py on pto

```
a = 'py' + 'thon'
a[0] = "a"
```

(result)

Traceback (most recent call last):

File "COMPSYS302\ 2_11_StringOperations.py",
line 2, in <module>

```
a[0] = "a"
```

TypeError: 'str' object does not support item
assignment

2.3 Operations

□ Sequence Operations: Sets

- Set objects are **immutable**, so attempting to rebind or delete an item or slice of a set raises an exception.

- Ex) 2_12_SetOperations.py

```
a = {1, 2, 3}
b = {3, 4, 5}
print (a, b)
print (a.union(b), a|b)      # union
print (a.intersection(b), a&b) # intersection
print (a-b)                 # difference set
```

(result)

{1, 2, 3} {3, 4, 5}

{1, 2, 3, 4, 5} {1, 2, 3, 4, 5}

{3} {3}

{1, 2}

2.3 Operations

□ Sequence Operations: Tuples

- Tuple objects are **immutable**, so attempting to rebind or delete an item or slice of a tuple raises an exception.

- Ex) 2_13_TupleOperations.py

```
t = (1, 2, 3, 1)
print (t)
print (type(t))
print (t.count(1))
```

```
(a, b) = (3, 4)
print (a, b)
```

```
a, b = b, a      # swap
print (a, b)
```

```
(a, b) = (b, a)  # swap
print (a, b)
```

```
(result)
(1, 2, 3, 1)
<type 'tuple'>
2
```

```
3 4
```

```
4 3
```

```
3 4
```

2.3 Operations

□ Sequence Operations: Lists

- List objects are **mutable**, so you may rebind or delete an item or slice of a list.

- Ex) 2_14_ListOperations.py

```
colours = ['red', 'green', 'gold']
print (colours, type(colours))
colours.append('blue'); print (colours)
colours.insert(1, 'black'); print (colours)
colours.extend(['white', 'gray']); print (colours)
colours += ['red']; print (colours)
colours += 'red'; print (colours)
print (colours.index('red'))
print (colours.index('red', 1))
print (colours.count('red'))
colours.pop(); print (colours)
colours.pop(1); print (colours)
colours.remove('red'); print (colours)
colours.sort(); print (colours)
colours.reverse(); print (colours)
```

(result)

```
['red', 'green', 'gold'] <type 'list'>
['red', 'green', 'gold', 'blue']
['red', 'black', 'green', 'gold', 'blue']
['red', 'black', 'green', 'gold', 'blue', 'white', 'gray']
['red', 'black', 'green', 'gold', 'blue', 'white', 'gray', 'red']
['red', 'black', 'green', 'gold', 'blue', 'white', 'gray', 'red', 'r', 'e', 'd']
0
7
2
['red', 'black', 'green', 'gold', 'blue', 'white', 'gray', 'red', 'r', 'e']
['red', 'green', 'gold', 'blue', 'white', 'gray', 'red', 'r', 'e']
['green', 'gold', 'blue', 'white', 'gray', 'red', 'r', 'e']
['blue', 'e', 'gold', 'gray', 'green', 'r', 'red', 'white']
['white', 'red', 'r', 'green', 'gray', 'gold', 'e', 'blue']
```

2.3 Operations

□ Sequence Operations: Dictionaries

- Python provides a variety of operations that can be applied to dictionaries.
- Dictionary objects provide several methods.

- Ex) 2_15_DictionaryOperations.py

```
colours = {'apple':'red', 'banana':'yellow'}  
print (colours, type(colours))  
print (colours['apple'])      # get value  
colours['cherry'] = 'red'     # add  
print (colours)  
colours['apple'] = 'green'    # update value  
print (colours)
```

(result)

{'apple': 'red', 'banana': 'yellow'} <type 'dict'>

red

{'apple': 'red', 'banana': 'yellow', 'cherry': 'red'}

{'apple': 'green', 'banana': 'yellow', 'cherry': 'red'}